arXiv:2312.12645v1 [stat.ME] 19 Dec 2023

Revisiting the effect of greediness on the efficacy of exchange algorithms for generating exact optimal experimental designs

William T. Gullion and Stephen J. Walsh
Utah State University

December 21, 2023

Abstract

Coordinate exchange (CEXCH) is a popular algorithm for generating exact optimal experimental designs. The authors of CEXCH advocated for a highly greedy implementation - one that exchanges and optimizes single element coordinates of the design matrix. We revisit the effect of greediness on CEXCHs efficacy for generating highly efficient designs. We implement the single-element CEXCH (most greedy), a design-row (medium greedy) optimization exchange, and particle swarm optimization (PSO; least greedy) on 21 exact response surface design scenarios, under the D- and $I-$criterion, which have well-known optimal designs that have been reproduced by several researchers. We found essentially no difference in performance of the most greedy CEXCH and the medium greedy CEXCH. PSO did exhibit better efficacy for generating D-optimal designs, and for most I-optimal designs than CEXCH, but not to a strong degree under our parametrization. This work suggests that further investigation of the greediness dimension and its effect on CEXCH efficacy on a wider suite of models and criterion is warranted.

Keywords: optimal design; exact design; coordinate exchange; greediness; particle swarm optimization

1 Introduction

[MN95] published the coordinate exchange (CEXCH) algorithm which enabled efficient generation of *exact* optimal experimental designs (DoE) with fully continuous factor levels. Based on previous studies, the authors suggested that algorithm performance increased as the algorithm's degree of greediness increased. Greedy in this sense means exchanging individual scalar elements of a design matrix, seeking a local optimizer for each coordinate in sequence, as opposed to optimizing multiple design coordinates, full design points, or all elements of the design in an effort to find the global optimizer. In this paper we benchmark two versions of exchange algorithms each representing 2 different levels of greediness, on 21 well-known exact optimal design scenarios under two optimality criterion. We compare the results to previously published work which used particle swarm optimization (PSO) to generate optimal designs [Wal21, WB22a]. PSO is a fully non-greedy optimizer that has performed very well in generating optimal designs for various scenarios over the last decade and is rapidly gaining in popularity for solving optimal DoE problems.

2 Survey of Algorithms for Generating Exact Optimal Designs

The following sections contain a brief literature survey of two main types of algorithms researchers appeal to for generating optimal DoEs: exchange algorithms and meta-heuristics.

2.1 Exchange Algorithms

The CEXCH of [MN95] appears to be the most popular optimal design algorithm for both application and academic research [GJ11, MN95, HN18, RJBM10, SP16, SAMJ23]. This popularity, in part, appears to be due to CEXCHs simplicity and ease of implementation. CEXCH proceeds by generating a completely random starting design. Each design coordinate is exchanged individually via some mechanism and the exchange that improves the optimality criterion the most is retained. The result is a *locally* (relative to the random starting point) optimal design. To increase the chances of finding either a highly efficient design or the globally optimal design, several authors recommend applying CEXCH several thousand times to each design problem [GJ11].

2.2 Particle Swarm Optimization

In contrast to CEXCH, PSO attempts to search the space of candidate designs more globally [Wal21, WB22a, WB22b]. PSO tends to cost more than CEXCH but is highly robust to entrapment in local optima and therefore can have a higher probability of find the globally optimal design. PSO is a relative newcomer to optimal design. [CHHW13] discuss PSO-generated latin-hypercube designs. [CHHW14, MJ18] use PSO to generate space-filling designs. [LMW19, CCW$^+$15, CLHW19] apply PSO to find optimal designs for non-linear models. PSO-generation of optimal designs for mixture experiments is provided in [WCHW15]. [CCW22] provide a comprehensive review of PSO

for generating continuous optimal designs. The efficacy of PSO for generating exact optimal designs was demonstrated in [WB22a]. New G-optimal designs were efficiently generated for small-exact response surface scenarios by an adaption of PSO with the local-communication topology [WB22b]. [WLAC23] used PSO to great effect to study the trade-offs of the G- and I-criterion (prediction variance designs) via a multi-objective optimization. A comprehensive review of PSO for solving the exact optimal design problem is provided in [Wal21].

3 Optimal Design for Small-Exact Response Surface Scenarios

We denote an exact N-run (rows) experimental design as $\mathbf{X} = \begin{pmatrix}\mathbf{x}_1 & \mathbf{x}_2 & \dots & \mathbf{x}_N\end{pmatrix}'$. Each row $\mathbf{x}_i'$ contains the settings of K (columns) experimental factors. As in RSM, we consider the standardized design space, the hypercube, denoted $\mathbf{x}_i' \in \mathcal{X} = [-1, 1]^K$ [MMA16]. We consider the second-order linear model which has $p = \binom{K+2}{2}$ linear coefficient parameters. In scalar form, the second-order model is written $y = \beta_0 + \sum_{i=1}^{K} \beta_i x_i + \sum_{i=1}^{K-1} \sum_{j=i+1}^{K} \beta_{ij} x_i x_j + \sum_{i=1}^{K} \beta_{ii} x_i^2 + \epsilon$. Let the $N \times p$ matrix $\mathbf{F}$ represent the model matrix with rows given by the expansion vector $\mathbf{f}'(\mathbf{x}_i') = (1 \quad x_{i1} \quad \dots \quad x_{i2} \quad x_{i1}x_{i2} \quad \dots \quad x_{i(K-1)}x_{iK} \quad x_{i1}^2 \quad \dots \quad x_{iK}^2)$. Note that $\mathbf{F}$ is an explicit function of design $\mathbf{X}$. Thus, the linear model is $\mathbf{y} = \mathbf{F}\boldsymbol{\beta} + \boldsymbol{\epsilon}$ where we assume $\boldsymbol{\epsilon} \sim \mathcal{N}_N(\mathbf{0}, \sigma^2 \mathbf{I}_N)$. The ordinary least squares estimator of $\boldsymbol{\beta}$ is $\hat{\boldsymbol{\beta}} = (\mathbf{F}'\mathbf{F})^{-1}\mathbf{F}'\mathbf{y}$ which has variance $\mathrm{Var}(\hat{\boldsymbol{\beta}}) = \sigma^2(\mathbf{F}'\mathbf{F})^{-1}$. The total information matrix for $\boldsymbol{\beta}$, $\mathbf{M}(\mathbf{X}) = \mathbf{F}'\mathbf{F}$, plays an important role in optimal DoE—all optimal design objective functions are functions of this matrix. The practitioner must choose a design from $\mathcal{X}^N$ to implement. An optimality criterion f is used score candidate design quality. An optimization algorithm is required to search $\mathcal{X}^N$ under f for high-quality designs. In this paper we consider the D- and I-criterion described in the next sections.

3.1 D-optimal Designs

The D-score of an arbitrary design $\mathbf{X}$ is the deteriminant of the inverse of the information matrix $D(\mathbf{X}) = |(\mathbf{F}'\mathbf{F})^{-1}|$. Thus the D-optimal design is $\mathbf{X}_D^* := \mathrm{argmin}_{\mathbf{X} \in \mathcal{X}^N} D(\mathbf{X})$. D-optimal designs are popular choices for screening experiments, and appear to be the most widely studied optimal DoE criterion [MN95, JN83, GJ11, ADT07, SAMJ23].

3.2 I-optimal Designs

I-optimal designs are those that minimize the average prediction variance over $\mathcal{X}$. The scaled prediction variance is often defined as $\mathrm{SPV}(\mathbf{x}'|\mathbf{X}) = N\mathbf{f}'(\mathbf{x}')(\mathbf{F}'\mathbf{F})^{-1}\mathbf{f}(\mathbf{x}')$ [RJBM10, Bor03]. The I-criterion for candidate design $\mathbf{X}$ is defined as $I(\mathbf{X}) = \frac{1}{V}\int_{\mathcal{X}} \mathrm{SPV}(\mathbf{x}'|\mathbf{X})\mathbf{d}\mathbf{x}' = \frac{N}{V}\mathrm{tr}\left\{(\mathbf{F}'\mathbf{F})^{-1}\int_{\mathcal{X}} \mathbf{f}(\mathbf{x}')\mathbf{f}'(\mathbf{x}')\mathbf{d}\mathbf{x}'\right\}$ where $V = \int_{\mathcal{X}} \mathbf{d}\mathbf{x}'$ yields the volume of the design space $\mathcal{X}$. The I-optimal design is defined as: $\mathbf{X}_I^* := \mathrm{argmin}_{\mathbf{X} \in \mathcal{X}^N} I(\mathbf{X})$. I-optimal designs are popular RSM designs due to their focus on giving predictions with minimized variance.

3.3 Second-Order Model: Best-Known Exact Optimal Designs

[Bor03] applied a Genetic Algorithm (GA) to generate candidate exact optimal designs for 21 distinct design scenarios (all second-order model) for $K = 1$ ($N = 3, 4, 5, 6, 7, 8, 9$), $K = 2$ ($N = 6, 7, 8, 9, 10, 11, 12$), and $K = 3$ ($N = 10, 11, 12, 13, 14, 15, 16$) design factors and four optimality criteria: the D-, A-, I-, and G-criterion. These designs have been reproduced by several authors, see e.g. [RJBM10, SP16, HN18, Wal21, WB22a, WB22b, WLAC23]. Thus, the catalog proposed by [Bor03] has become a domain-standard dataset of exact optimal designs that is highly used for algorithm benchmarking. We adopt these designs as ground-truth for the purpose of this study.

4 Implemented Algorithms

For each of the 21 design scenarios in [Bor03], we ran an element-wise (fully greedy) and a row-wise (less greedy) EXCH algorithm $n_{\text{run}} = 10,000$ independent times. For PSO, the most non-greedy algorithm in this comparative study, we take advantage of a previously published benchmarking database for these specific design scenarios presented and described in [Wal21, WB22a] who published performance data on $n_{\text{run}} = 140$ independent PSO runs. These authors demonstrated high efficacy of PSO even with this number of runs. Brief descriptions of the CEXCH algorithms follow.

4.1 Element-wise CEXCH

This is the most greedy algorithm in this study. Early versions of CEXCH used the *addition*, *deletion*, and *delta* update functions which identify candidate coordinates to add/delete and measure the effect of these changes on the candidate optimal design [JN83, MN95, SAMJ23]. These update functions, however, are only mathematically tractable for simpler design problems and optimality criteria, such as the D- and A-criterion [MN95, SAMJ23]. Therefore, it is common practice to implement CEXCH by adapting a univariate optimization algorithm, as shown in line 7 of Algorithm 1, as the proposal mechanism. For example, authors [RJBM10, GJ11] apply Brent's minimization algorithm as the mechanism for proposing coordinate exchanges. In our code, we used the `BOBYQA` optimization algorithm (univariate) in `Julia`'s `NLOpt` package [BEKS17, Joh]. In line 7, x_{ij} represents the ijth element of candidate design $\mathbf{X}$, while $\mathbf{X}_{-ij}$ represents the candidate design *without* the ijth element. Thus the univariate optimization attempts to find the value x_{ij} that improves the criterion conditioned on all other design coordinates $\mathbf{X}_{-ij}$ remaining fixed. The procedure proceeds by looping through the rows and columns of $\mathbf{X}$ until a full pass over the NK design coordinates is conducted with no improvements found.

4.2 Row-wise EXCH

This algorithm represents the middle-ground of greediness for all algorithms in this study. [SAMJ23] provides some discussion of row-exchange algorithms and their application/tractability on optimal design problems. In this algorithm, one only loops through the rows of design $\mathbf{X}$ and exchanges (under some proposal

Algorithm 1 Element-wise CEXCH Pseudocode

1: **Inputs:** $K :=$ number of experimental factors, $N :=$ number of affordable experimental runs, $f :=$ the optimal design criterion

2: // Randomly instantiate an $N \times K$ design matrix from a uniform distribution

3: $\mathbf{X} \leftarrow \{x_{ij} \overset{i.i.d}{\sim} U(-1,1) \text{ for } i = 1, \ldots, N, j = 1, \ldots, K\}$

4: **while** $\{$*improvements to f are found* $\}$ **do**

5: **for** $i = 1, 2, \ldots, N$ **do**

6: **for** $j = 1, 2, \ldots, K$ **do**

7: $\mathbf{X} \leftarrow \text{argmax}_{x_{ij} \in [-1,1]} f(x_{ij} | \mathbf{X}_{-ij})$ // coordinate proposal is a univariate optimization

8: **endfor**

9: **endfor**

10: **endwhile**

11: **Output:** $\mathbf{X} :=$ a locally optimal design

mechanism) a K-variate design point $\mathbf{x}'_i$ in an effort to improve the candidate design's criterion score. In Line 6 of Algorithm 2, we implement the K-variate `BOBYQA` optimization algorithm from the `NLOpt` package of `Julia`. The algorithm proceeds by looping through the rows of $\mathbf{X}$ and repeating the K-variate optimization until a full pass over the N design points is conducted with no improvements found.

Algorithm 2 Row-wise EXCH Pseudocode

1: **Inputs:** $K :=$ number of experimental factors, $N :=$ number of affordable experimental runs, $f :=$ the optimal design criterion

2: // Randomly instantiate an $N \times K$ design matrix from a uniform distribution

3: $\mathbf{X} \leftarrow \{x_{ij} \overset{i.i.d}{\sim} U(-1,1) \text{ for } i = 1, \ldots, N, j = 1, \ldots, K\}$

4: **while** $\{$*improvements to f are found* $\}$ **do**

5: **for** $i = 1, 2, \ldots, N$ **do**

6: $\mathbf{X} \leftarrow \text{argmax}_{\mathbf{x}'_i \in [-1,1]^K} f(\mathbf{x}'_i | \mathbf{X}_{-i})$ // design-point proposal is a K-variate optimization

7: **endfor**

8: **endwhile**

9: **Output:** $\mathbf{X} :=$ a locally optimal design

5 Results

We summarize the efficacy of algorithm performance in two measures: (1) $\hat{\Pr}$(*generating a 95% efficient or better design*) and (2) $\hat{\Pr}$(*generating the best-known design*) similar to data presented by [WB22a]. Estimates of each probability are computed as the proportion of times in n_{run} we observed the success result in our data.

First we briefly summarize Figure 1 which reports probability estimates for each algorithm generating a 95% efficient or better design. Rows of the graph panel represent algorithm, columns represent K (# design factors), and the horizontal axis represents N, the number of experimental runs. The black dotted line represents probability estimates for generating I optimal designs. We essentially observe no difference in performance between any of the algorithms for generating I-optimal design over all $\{K, N\}$ design scenarios. Switching to D-optimal designs and the grey curve, we can SEE PSO demonstrates appreciably larger probability of finding highly efficient designs for the $K = 1, 2$ scenarios, but not for $K = 3$, indicating PSO would need stronger parameters to realize a benefit. Last, we note that element-wise CEXCH (least-greedy) and row-wise (medium greedy) appear to exhibit the same efficacy, a result that seems in-contra to the results of [MN95].

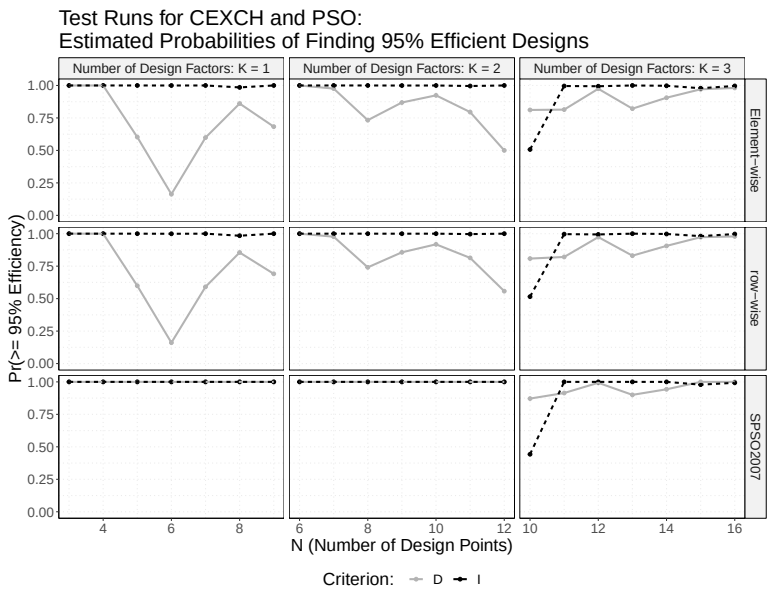

Figure 1: Data driven estimates that each algorithm generates a 95% efficient or better design in single run for each scenario.

Next, we inspect each algorithms efficacy in finding the globally optimal design and present probability of success estimates in Figure 2. First, inspecting the black curve, I-optimal design searches, for $K = 1$ PSO found the globally optimal design each time, but each CEXCH found it with high probability. PSO appears to have a small edge on the EXCH algorithms for K= 2, and for K=3, PSO does better for some N but not for others. Last, the results of element-wise and row-wise CEXCH appear to be essentially the same. Next, switching to the grey curves for D-optimal design searches, PSO does appreciably better than the EXCH algorithms in finding the globally optimal design for the $K = 1, 2$ scenarios, and only slightly better for the $K = 3$ scenarios. Again, we observe no appreciable difference in performance between the most greedy and less greedy exchange algorithms.

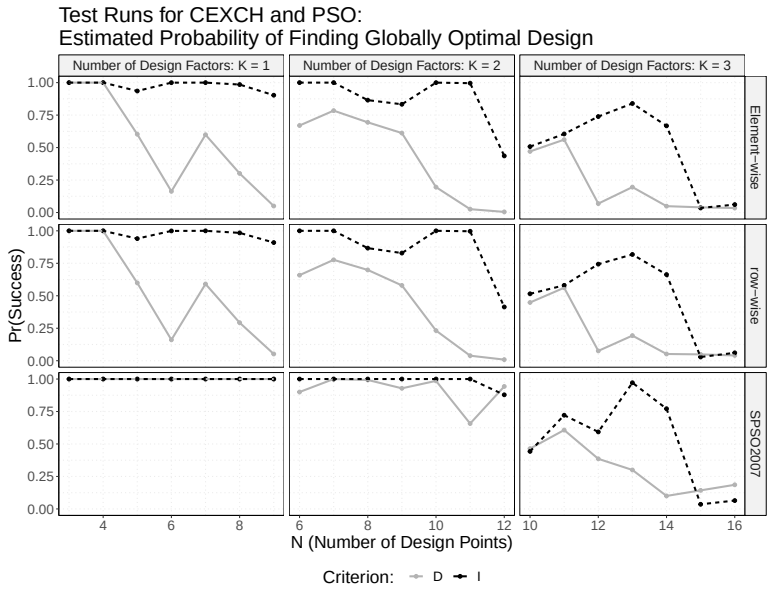

Figure 2: Data driven estimates that each algorithm generates the globally optimal design in single run for each scenario.

6 Conclusions and Future Work

In this study we apparently found results somewhat in contra to the initial observations and motivations of [MN95]. There may be several explanations. First, [MN95] provided benchmarking data comparing CEXCH to the older PEXCH algorithms which utilize a pre-defined candidate set and apparently only published a time-cost comparison. In addition, [MN95] apparently only benchmarked D-optimal design runs, and it seems necessary that such benchmarking be conducted over several optimal design objectives which have different characteristics. Further, when comparing the quality of designs returned by the two algorithms, [MN95] (pp.64 §3) state *"...Although it is common practice to run a design algorithm from several starting designs, each test case in the current study was run just once."* To our view it is necessary to study and benchmark algorithm performance on many independent runs of the algorithm. Seemingly in contrast to [MN95], the most greedy element-wise and less greey row-wise EXCH algorithms appear to have the smae performance when using optimizers as the exchange algorithm. This result should be investigated further. Further, any algorithm appeard to do an equally good job at generating I-optimal designs, but the differences expected between PSO and CEXCH were more apparent with the D-criterion. This suggests that, perhaps, the I-criterion is not so fraught with local optima as the D-optimal designs may be. Another result that should be investigated further.

These results suggests that our community should revisit the greediness dimension in algorithms for generating optimal designs. In future work we intend to expand this study in several dimensions: 1) a more comprehensive model set, including first-order, interaction, linear and non-linear models, 2) larger

experiments (i.e. increasing K), and 3) a broader set of optimal design criteria including A- and $G-$criterion, and 4) inclusion of additional versions of CEXCH and other metaheursitics that have been used for generating optimal DoEs (sp. GA and SA).

References

[ADT07] A.C. Atkinson, A.N. Donev, and R.D. Tobias. *Optimal Experimental Designs, with SAS*. Oxford Statistical Science Series, 2007.

[BEKS17] Jeff Bezanson, Alan Edelman, Stefan Karpinski, and Viral B Shah. Julia: A fresh approach to numerical computing. *SIAM review*, 59(1):65–98, 2017.

[Bor03] John J. Borkowski. Using a genetic algorithm to generate small exact response surface designs. *Journal of Probability and Statistical Science*, 1(1), Feb. 2003.

[CCW+15] Ray-Bing Chen, Shin-Perng Chang, Weichung Wang, Heng-Chih Tung, and Weng Kee Wong. Minimax optimal designs via particle swarm optimization methods. *Statistics and Computing*, 25(5):975–988, Sep 2015.

[CCW22] Ping-Yang Chen, Ray-Bing Chen, and Weng Kee Wong. Particle swarm optimization for searching efficient experimental designs: A review. *WIREs Computational Statistics*, 14(5):e1578, 2022.

[CHHW13] Ray-Bing Chen, Dai-Ni Hsieh, Ying Hung, and Weichung Wang. Optimizing latin hypercube designs by particle swarm. *Statistics and Computing*, 23(5):663–676, Sep 2013.

[CHHW14] R-B. Chen, Y-W. Hsu, Y. Hung, and W. Wang. Discrete particle swarm optimization for constructing uniform design on irregular regions. *Computational Statistics and Data Analysis*, 72:282–297, 2014.

[CLHW19] Ray-Bing Chen, Chi-Hao Li, Ying Hung, and Weichung Wang. Optimal noncollapsing space-filling designs for irregular experimental regions. *Journal of Computational and Graphical Statistics*, 28(1):74–91, 2019.

[GJ11] P. Goos and B. Jones. *Optimal Design of Experiments: A Case study approach.* John Wiley and Sons, Ltd., 2011.

[HN18] Lucia N. Hernandez and Christopher J. Nachtsheim. Fast computation of exact G-optimal designs via I_λ-optimality. *Technometrics*, 60(3):297–305, 2018.

[JN83] Mark E. Johnson and Christopher J. Nachtsheim. Some guidelines for constructing exact D-optimal designs on convex design spaces. *Technometrics*, 25(3):271–277, 1983.

[Joh] S.G. Johnson. The nlopt nonlinear-optimization package.

[LMW19] Joshua Lukemire, Abhyuday Mandal, and Weng Kee Wong. d-qpso: A quantum-behaved particle swarm technique for finding D-optimal designs with discrete and continuous factors and a binary response. *Technometrics*, 61(1):77–87, 2019.

[MJ18] Simon Mak and V. Roshan Joseph. Minimax and minimax projection designs using clustering. *Journal of Computational and Graphical Statistics*, 27(1):166–178, 2018.

[MMA16] R. Myers, D. Montgomery, and C. Anderson-Cook. *Response surface methodology: process and product optimization using Designed Experiments. 4th Edition.* John Wiley and Sons, Ltd., 2016.

[MN95] Ruth K. Meyer and Christopher J. Nachtsheim. The coordinate-exchange algorithm for constructing exact optimal experimental designs. *Technometrics*, 37(1):60–69, 1995.

[RJBM10] Myrta Rodríguez, Bradley Jones, Connie M. Borror, and Douglas C. Montgomery. Generating and assessing exact G-optimal designs. *Journal of Quality Technology*, 42(1):3–20, 2010.

[SAMJ23] Jonathan Stallrich, Katherine Allen-Moyer, and Bradley Jones. D- and a-optimal screening designs. *Technometrics*, 0(0):1–10, 2023.

[SP16] Moein Saleh and Rong Pan. A clustering-based coordinate exchange algorithm for generating g-optimal experimental designs. *Journal of Statistical Computation and Simulation*, 86(8):1582–1604, 2016.

[Wal21] S. J. Walsh. *Development and applications of particle swarm optimization for constructing optimal experimental designs.* Ph.D. Thesis. Montana State University., 2021.

[WB22a] Stephen J. Walsh and John J. Borkowski. Generating exact optimal designs via particle swarm optimization: Assessing efficacy and efficiency via case study. *Quality Engineering*, 35(2):304–323, 2022.

[WB22b] Stephen J. Walsh and John J. Borkowski. Improved g-optimal designs for small exact response surface scenarios: Fast and efficient generation via particle swarm optimization. *Mathematics*, 10(22), 2022.

[WCHW15] Weng Kee Wong, Ray-Bing Chen, Chien-Chih Huang, and Weichung Wang. A modified particle swarm optimization technique for finding optimal designs for mixture models. *PLOS ONE*, 10(6):1–23, 06 2015.

[WLAC23] Stephen J. Walsh, Lu Lu, and Christine M. Anderson-Cook. I-optimal or g-optimal: Do we have to choose? *Quality Engineering*, 0(0):1–22, 2023.